

Credit Default Risk

Model Training & Evaluation — Technical Report

Detailed analysis of the notebook **building_models.ipynb**: data pipeline, modeling approach, experiments, and results.

Field	Value
Notebook	building_models (1).ipynb
Task	Binary classification — predict loan/credit default
Data source	karimabdelmonem/automated-explainable-ai-system (GitHub, processed data)
Models trained	10 (1 linear, 3 tree-based + 2 tuned variants, 2 deep learning, 1 SMOTE variant, 1 ensemble)
Best model (ROC-AUC)	LightGBM (default params) — ROC-AUC 0.7428
Best model (F1, tuned threshold)	Optimized MLP (Tabular_DL_Optimized) — F1 0.8217 @ threshold 0.228
Train / Test / Val sizes	304,952 / 38,172 / 38,171 rows — 36 features

1. Introduction & Objective

This notebook builds and rigorously evaluates multiple machine learning model families — linear, tree-based ensemble, and deep learning — on a **credit/loan default classification task**. The goal is to predict whether a borrower will default (target column **default**, 1 = default, 0 = no default) using a pre-engineered tabular feature set.

Beyond simply training models, the notebook layers on a full evaluation methodology: hyperparameter tuning, class-imbalance handling (class-weighting and SMOTE), stratified cross-validation, learning-curve diagnostics, and a per-model decision-threshold optimization step — all aimed at producing an honest, imbalance-aware picture of model quality rather than relying on a single, potentially misleading accuracy number.

Pipeline at a Glance

- **Setup** — install dependencies, download processed data files from GitHub.
 - **Baseline model** — regularized, class-balanced Logistic Regression.
 - **Tree-based models** — Random Forest, XGBoost, LightGBM, each with built-in imbalance handling.
 - **Deep learning** — two MLP (multi-layer perceptron) architectures of increasing depth.
 - **Hyperparameter tuning** — RandomizedSearchCV on LightGBM, plus a validation-based early-stopping variant.
 - **Imbalance strategy comparison** — SMOTE oversampling vs. implicit class-weighting.
 - **Ensembling** — soft-voting classifier combining LightGBM, XGBoost, and Random Forest.
 - **Cross-validation** — stratified 10-fold CV for a generalization check.
 - **Diagnostics** — classification reports, confusion matrices, learning curves, precision-recall curves.
 - **Threshold optimization** — per-model F1-optimal decision threshold, replacing the naive 0.5 cutoff.
 - **Final leaderboard** — ranks all 10 models both at the default threshold and at their tuned threshold.
-

2. Data Acquisition & Description

2.1 Source

The notebook installs the single missing dependency (`imbalanced-learn`) and then downloads six pre-processed Parquet files directly from the **karimabdelmonem/automated-explainable-ai-system** GitHub repository (`data/processed/` folder) using HTTP requests: `X_train`, `X_test`, `X_val`, and their corresponding `y_train`, `y_test`, `y_val` label files. This means the notebook consumes data that was *already cleaned and feature-engineered* in an upstream project — no raw-data cleaning happens here.

File	Downloaded Size
X_train.parquet	34,188,497 bytes
X_test.parquet	4,472,453 bytes
X_val.parquet	4,473,752 bytes
y_train.parquet	24,773 bytes
y_test.parquet	6,446 bytes

File	Downloaded Size
y_val.parquet	6,459 bytes

2.2 Shape and Schema Alignment

After loading, the datasets have the following shapes:

Split	Rows	Columns
X_train	304,952 rows	36 columns
X_test	38,172 rows	36 columns
X_val	38,171 rows	36 columns

A schema mismatch was handled explicitly: the published X_val file originally carried a few extra engineered columns not present in X_train/X_test (e.g. risk_score, log_income). Since any model trained on the 36 training columns would raise an error if evaluated against a validation set with a different column set, the notebook re-aligns X_val to exactly match X_train.columns before it is used anywhere downstream (e.g. as the early-stopping eval set for LightGBM).

2.3 Class Balance — a Critical Detail

The target variable (default) is **not** balanced the same way across splits, and this drives most of the notebook's later methodology:

Split	No-Default (0)	Default (1)	Note
y_train	50.0%	50.0%	Artificially balanced (oversampled)
y_test	88.4%	11.6%	Real-world distribution
y_val	~88%	~12%	Real-world distribution

Why this matters: because y_train is perfectly balanced while y_test/y_val reflect the true ~88%/12% imbalance, a trivial model that always predicts "no default" would already score ~88% accuracy on the test set. This makes **accuracy alone an unreliable metric** throughout the notebook — F1-score, ROC-AUC, and the threshold-tuned metrics introduced in Section 15 (of the notebook) are weighted much more heavily in the analysis.

3. Modeling Methodology & Results

3.1 Linear Baseline — Logistic Regression

A Pipeline of StandardScaler + LogisticRegression (class_weight='balanced', max_iter=2000) is trained as the baseline. It is saved as `logistic_regression_pipeline.joblib`.

Model	Accuracy	F1-Score	ROC-AUC
Logistic Regression	0.8198	0.8257	0.6801

3.2 Tree-Based Models (Imbalance-Aware)

Random Forest, XGBoost, and LightGBM are trained with imbalance handling built directly into their hyperparameters — `class_weight='balanced'` for Random Forest and LightGBM, and `scale_pos_weight` for XGBoost (computed as the ratio of negative to positive class counts; here it evaluates to 1.0000 since `y_train` is already 50/50). Each model is saved individually as a `.joblib` file.

Model	Key Hyperparameters	Accuracy	F1-Score	ROC-AUC
Random Forest	n_estimators=300, max_depth=12, class_weight=balanced	0.8185	0.8245	0.7138
XGBoost	n_estimators=500, lr=0.03, scale_pos_weight=1.0	0.8806	0.8484	0.7394
LightGBM	n_estimators=500, lr=0.03, class_weight=balanced	0.8822	0.8483	0.7428

At this stage, LightGBM already emerges as the strongest model by ROC-AUC (0.7428), narrowly ahead of XGBoost (0.7394), while the plain Random Forest lags behind both boosting methods.

3.3 Deep Learning — Baseline MLP

A feed-forward neural network (Keras/TensorFlow) is trained as a fourth model family: Dense(128) → BatchNorm → Dropout(0.3) → Dense(64) → BatchNorm → Dropout(0.2) → Dense(32) → Dropout(0.1) → Dense(1, sigmoid). It uses the Adam optimizer (lr=0.001), binary cross-entropy loss, a 20% validation split, and callbacks for early stopping (patience=5) and learning-rate reduction on plateau. Training ran for 11 epochs before early stopping triggered (of a max 20), reaching a validation loss of ~0.133.

Model	Accuracy	F1-Score	ROC-AUC
Tabular DL (baseline MLP)	0.8701	0.8450	0.7424

3.4 Deep Learning — Optimized MLP

A deeper, more heavily regularized architecture is trained independently (not replacing the baseline, so the two remain comparable): Dense(512) → BatchNorm → Dropout(0.40) → Dense(256) → BatchNorm → Dropout(0.30) → Dense(128) → BatchNorm → Dropout(0.25) → Dense(64) → Dense(1, sigmoid). It uses a lower learning rate (0.0005), longer patience (10 epochs), and a larger batch size (128) over up to 50 epochs.

Model	Accuracy	F1-Score	ROC-AUC
Tabular DL Optimized	0.8767	0.8477	0.7413

ROC curves for all models trained so far are plotted together and saved as `roc_curves_baseline_models.png`.

3.5 Hyperparameter Tuning — LightGBM (RandomizedSearchCV)

RandomizedSearchCV (15 iterations, 3-fold CV, optimizing ROC-AUC) searches over a grid of `n_estimators`, `learning_rate`, `num_leaves`, `max_depth`, and `class_weight`. The best cross-validated ROC-AUC found during the search was **0.9565**, with best parameters: `num_leaves=128`, `n_estimators=300`, `max_depth=20`, `learning_rate=0.1`, `class_weight=None`.

Model	Accuracy	F1-Score	ROC-AUC
LightGBM Tuned	0.8808	0.8477	0.7332

Note the gap between the CV search score (0.9565, measured on the balanced 50/50 `y_train` via 3-fold CV) and the held-out test-set ROC-AUC (0.7332, measured on the real-world imbalanced `y_test`). This is expected and again illustrates why in-sample or balanced-CV scores can look far more optimistic than genuine real-world performance.

3.6 Validation-Based Early Stopping for the Tuned LightGBM

The tuned hyperparameters are reused to retrain LightGBM, this time with a dedicated validation set (`X_val/y_val`) and early stopping (20 rounds patience) on binary log-loss. Training stopped early at iteration 125 (best validation binary log-loss ≈ 0.3228).

Resulting classification report on the test set (threshold = 0.5):

Class	Precision	Recall	F1-Score	Support
No-Default (0)	0.89	0.98	0.94	33,744
Default (1)	0.45	0.12	0.19	4,428
Accuracy			0.88	38,172

This model is saved as `lightgbm_val_earlystop_model.joblib`.

3.7 Class-Imbalance Strategy Comparison — SMOTE vs. Class-Weighting

To test whether explicit oversampling adds value beyond implicit class-weighting, SMOTE (Synthetic Minority Over-sampling Technique) is applied to `X_train/y_train`, and the tuned LightGBM hyperparameters (excluding `class_weight`) are retrained on the resampled data. Since `y_train` was already perfectly balanced (50/50) before SMOTE, the resampled shape is unchanged at (304,952, 36).

Model	Accuracy	F1-Score	ROC-AUC
LightGBM Tuned (class-weighted)	0.8808	0.8477	0.7332
LightGBM SMOTE	0.8808	0.8477	0.7332

Key finding: the two approaches produce identical metrics. Because `y_train` was already artificially balanced upstream, SMOTE had no additional class-imbalance signal left to correct — it neither helped nor hurt. This is a genuinely informative negative result: further oversampling on top of an already-balanced training set is redundant here.

3.8 Ensemble — Soft Voting Classifier

A `VotingClassifier` (soft voting, i.e. probability-averaging) combines the tuned `LightGBM`, `XGBoost`, and `Random Forest` models.

Model	Accuracy	F1-Score	ROC-AUC
Ensemble (Soft Voting)	0.8744	0.8494	0.7346

The ensemble achieves the highest weighted F1-score among all default-threshold results (0.8494), though its ROC-AUC sits in the middle of the pack rather than at the top.

4. Diagnostics & Validation

4.1 Stratified 10-Fold Cross-Validation

A stratified 10-fold cross-validation (each fold preserving the 50/50 balance of `y_train`) is run for four representative models, tracking accuracy, F1, and ROC-AUC — not accuracy alone.

Model	Accuracy (mean $\pm$ std)	F1 (mean)	ROC-AUC (mean $\pm$ std)
Logistic Regression	0.8700 (± 0.0023)	0.8678	0.9359 (± 0.0017)
Random Forest	0.8724 (± 0.0024)	0.8707	0.9408 (± 0.0017)
XGBoost	0.9233 (± 0.0024)	0.9188	0.9639 (± 0.0015)
LightGBM Tuned	0.9278 (± 0.0019)	0.9238	0.9658 (± 0.0012)

These CV numbers (measured on the balanced `y_train` via cross-validation) are considerably higher than the held-out test-set numbers in Section 3, since the CV folds still reflect the artificial 50/50 balance rather than the real-world 88/12 split. LightGBM Tuned and XGBoost show the strongest and most stable (lowest std) cross-validated performance.

4.2 Classification Reports & Confusion Matrices — All Models

Full precision/recall/F1 classification reports (at the default 0.5 threshold) are printed for all 10 models, and confusion matrices for every model are plotted together in `confusion_matrices_all_models.png`. Across the board, precision and recall on the minority "Default" class are visibly weaker than on the majority class — consistent with the class-imbalance issue flagged in Section 2.3. For example, at the default threshold, XGBoost and LightGBM catch only ~11–12% recall on actual defaults despite ~88% overall accuracy.

4.3 Learning Curves — Bias / Variance Diagnostics

For runtime efficiency, a stratified subsample (up to 30,000 rows) is used to compute learning curves (3-fold CV, F1 scoring, 6 training-size steps) for Logistic Regression, Random Forest, XGBoost, and LightGBM Tuned, saved to `learning_curves_diagnostics.png`. These curves are used to check whether training and validation F1 scores converge (well-fit), plateau together at a low score (underfitting), or diverge with a persistent gap (overfitting).

4.4 Precision-Recall Curves — All Models

Combined precision-recall curves (with average precision, AP) for all 10 models are plotted against the no-skill baseline (the test-set positive rate, 11.6%) and saved as `precision_recall_curves_all_models.png`. This view is especially informative for imbalanced problems, where ROC curves can look deceptively strong.

5. Decision-Threshold Optimization

Every evaluation up to this point used the default 0.5 probability cutoff. Given the train/test class-balance mismatch, 0.5 is very unlikely to be optimal. This section sweeps the full threshold range for every model's predicted probabilities and selects the F1-maximizing threshold per model.

Model	Best Thr.	Accuracy (0.5→tuned)	F1 (0.5→tuned)	Tuned Precision (Default)	Tuned Recall (Default)
Tabular_DL_Optimized	0.228	0.8767→0.8019	0.8477→0.8217	0.2852	0.4697
Tabular_DL	0.261	0.8701→0.7886	0.8450→0.8134	0.2746	0.5011
LightGBM	0.208	0.8822→0.7858	0.8483→0.8113	0.2706	0.4991
LightGBM_Tuned	0.187	0.8808→0.7838	0.8477→0.8096	0.2665	0.4932
LightGBM_SMOTE	0.187	0.8808→0.7838	0.8477→0.8096	0.2665	0.4932
XGBoost	0.207	0.8806→0.7747	0.8484→0.8038	0.2607	0.5133
Ensemble_Soft_Voting	0.260	0.8744→0.7711	0.8494→0.8011	0.2557	0.5090
LightGBM_ValEarlyStop	0.181	0.8810→0.7670	0.8489→0.7991	0.2589	0.5413
Logistic_Regression	0.366	0.8198→0.7683	0.8257→0.7961	0.2308	0.4277
Random_Forest	0.365	0.8185→0.7345	0.8245→0.7751	0.2259	0.5307

Sorted by tuned (threshold-optimal) F1, the top result is:

Tabular_DL_Optimized is the strongest model once the threshold is tuned (F1 = 0.8217 at threshold 0.228), even though it did not have the top ROC-AUC at the default cutoff. As expected, tuned accuracy drops relative to the 0.5-threshold accuracy while tuned F1 and default-class recall rise substantially — this trade-off is intentional, since catching actual defaults is the real business objective in credit risk, not raw accuracy.

A worked example on the single best model by tuned F1 (Tabular_DL_Optimized) makes this concrete: at the default 0.5 threshold it catches only **572 of 4,428** actual defaults in the test set; at its F1-optimal threshold of 0.228, it catches **2,080 of 4,428** — nearly 3.6× more actual defaults identified, at the cost of more false positives. This comparison is visualized in `best_model_threshold_confusion_comparison.png`.

5.1 Accuracy vs. Recall Trade-off

A further diagnostic sweeps the full threshold range for the strongest model (Tabular_DL_Optimized) and plots accuracy, recall, and precision together against a hypothetical 95% accuracy target line, saved as `accuracy_recall_tradeoff.png`. The notebook's stated conclusions from this analysis:

- Peak achievable accuracy across *all* thresholds tops out at **88.40%**.
- The ROC-AUC ceiling for the strongest model is **0.7413**.
- A 95–98% accuracy target is **not supported** by the current signal-to-noise ratio in the dataset — reaching it would require near-perfect class separation (ROC-AUC → 1.0), which is far beyond what any model here achieves. Closing that gap would require new features or a reframed business objective, not further threshold or hyperparameter tuning.

6. Final Leaderboard

6.1 Default Threshold (0.5), Sorted by ROC-AUC

Model	Accuracy	F1-Score	ROC-AUC
LightGBM	0.8822	0.8483	0.7428
Tabular_DL	0.8701	0.8450	0.7424
Tabular_DL_Optimized	0.8767	0.8477	0.7413
XGBoost	0.8806	0.8484	0.7394
LightGBM_ValEarlyStop	0.8810	0.8489	0.7385
Ensemble_Soft_Voting	0.8744	0.8494	0.7346
LightGBM_Tuned	0.8808	0.8477	0.7332
LightGBM_SMOTE	0.8808	0.8477	0.7332
Random_Forest	0.8185	0.8245	0.7138
Logistic_Regression	0.8198	0.8257	0.6801

6.2 F1-Optimal Threshold per Model, Sorted by Tuned F1

Model	Best Threshold	Tuned Accuracy	Tuned F1
Tabular_DL_Optimized	0.2283	0.8019	0.8217
Tabular_DL	0.2606	0.7886	0.8134
LightGBM	0.2082	0.7858	0.8113
LightGBM_Tuned	0.1865	0.7838	0.8096
LightGBM_SMOTE	0.1865	0.7838	0.8096
XGBoost	0.2073	0.7747	0.8038
Ensemble_Soft_Voting	0.2595	0.7711	0.8011
LightGBM_ValEarlyStop	0.1811	0.7670	0.7991
Logistic_Regression	0.3662	0.7683	0.7961
Random_Forest	0.3645	0.7345	0.7751

All ten trained models, along with the threshold comparison table, are saved to disk: *.joblib for sklearn/XGBoost/LightGBM models, *.keras for the two neural networks, and optimal_threshold_comparison.csv for the threshold sweep results. In the original Colab environment, a final cell downloads all generated model files and plots to the local machine.

7. Summary & Recommendations

What the notebook covers, end to end: a regularized logistic regression baseline; imbalance-aware Random Forest, XGBoost, and LightGBM; two MLP architectures of different depth/regularization; a RandomizedSearchCV-tuned LightGBM with both a standard fit and a dedicated-validation-set early-stopping variant; a SMOTE-resampled LightGBM to compare explicit oversampling against implicit class-weighting; a soft-voting ensemble; stratified 10-fold cross-validation; full classification reports, confusion matrices, and learning curves for every model; combined precision-recall curves; and a per-model F1-optimal threshold sweep with an accuracy-vs-recall trade-off analysis.

Honest takeaway: the available features carry a real but moderate signal (ROC-AUC roughly in the 0.68–0.74 range on held-out test data across models), and the train/test class-balance mismatch means raw accuracy is not a trustworthy metric on its own. None of the techniques layered on top (SMOTE, ensembling, deeper architectures, extensive tuning) closed that gap by much — which is itself informative: the performance ceiling appears to be set by the *features*, not the *models*.

Recommended Next Steps (in order of expected payoff)

- **Engineer new features** — interaction terms, payment-history-derived ratios, or external bureau-style data — since the current ROC-AUC ceiling suggests real signal is being left on the table.
- **Recalibrate probabilities** with `CalibratedClassifierCV` if the model is deployed as a risk-scoring tool rather than a hard classifier.
- **Choose the operating threshold based on real business cost** — the cost of a missed default vs. a false alarm — using the trade-off curve from Section 5, rather than defaulting to 0.5.
- **Reframe as a ranking/scoring problem** if more recall is required — returning a risk percentile instead of a binary classification removes the threshold-choice problem entirely.